RC5 Encryption:

RC5 Modes of operation:

RC5 is a symmetric block cipher that can operate in several modes similar to other block ciphers:

(i) Electronic codebook (ECB) Mode: Each block is encrypted independently.

(ii) Cipher Block Chaining (CBC) Mode: Each plaintext block is XORed with the previous ciphertext block before encryption.

(iii) Cipher feedback (CFB) Mode: The cipher operates as a stream cipher, encrypting the previous ciphertext block.

(iv) Output feedback (OFB) Mode: Similar to CFB but uses the encrypted IV repeatedly.

(v) Counter (CTR) Mode: Encrypts a counter value to produce a key stream.

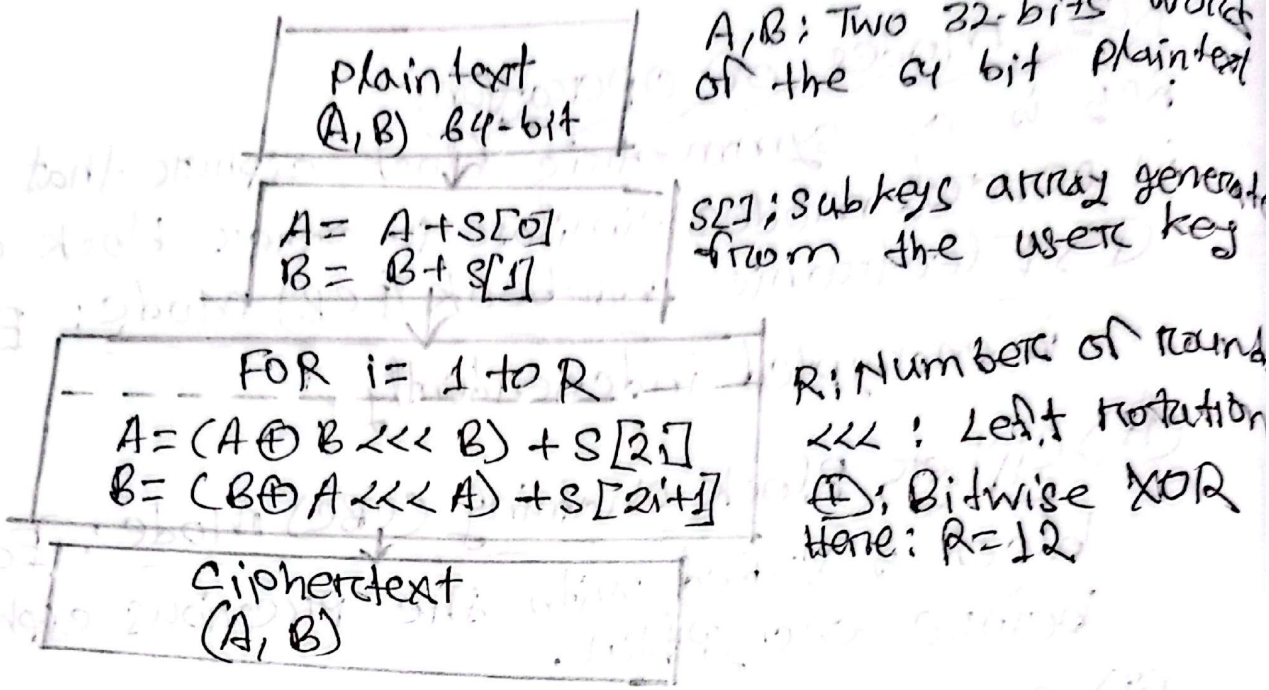
RC5 Block Diagram:

RC5 is a symmetric key block cipher notable for

* Variable word size(w): e.g., 32 bits
* Number of rounds (r): e.g., 12
* key size(b): e.g., 16 bytes (128 bits)

— P.T.O —

Encryption process:

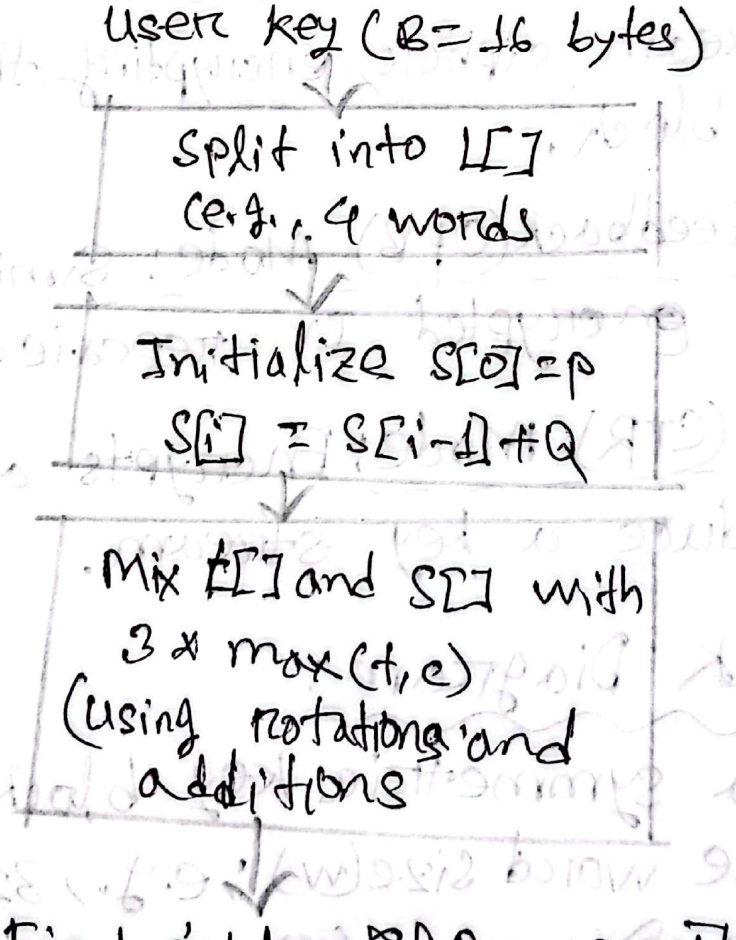


A, B: Two 32-bits words of the 64 bit plaintext

S[]: subkeys array generated from the user key

R: Number of rounds
<<< : Left rotation
⊕: Bitwise XOR
Here: R = 12

Key Expansion Diagram:



Java Implementation Highlights:

Key constants:

```
Private static final int W = 32; // word size
Private static final int R = 12; // Rounds
private static final int B = 16; // key size (bytes)
Private static final int P = 0XB7E15163; // Magic
Private static final int Q = 0X9E3779B9; constants
```

Core operations:

```
// Circular left rotation
Private static int rotl(int x, int y){
 return (x<<y) | (x>>>(W-y)); }
// key expansion
 Private static void rc5Key setup(byte[] key, int[] S){
  //... key expansion logic... }
// Encryption round
Private static void rc5Encrypt(int[] S, int[] data){
 // -- encryption logic... }
```

Sample Input/Output:

Input: "Hello RC5!"

Output: "a3d4f12e 5b678c90"